

1. Понятие системы. Общие представления о сложной системе.

Система — это ограниченное, взаимно связанное множество, отражающее объективное существование конкретных отдельных совокупностей объектов и не содержащее специфических ограничений, присутствующих в частной системе. Согласно источнику, **сложная система** обладает следующими **основными свойствами**:

- **Отграниченность:** система отделена от окружающей среды определенными границами.
- **Целостность:** это качественное свойство, при котором качества системы как целого принципиально не сводятся к простой сумме свойств её отдельных элементов.
- **Структурность:** поведение системы обусловлено не столько особенностями отдельных элементов, сколько специфическими свойствами её структуры.
- **Взаимозависимость со средой:** система формирует и проявляет свои свойства только в процессе постоянного взаимодействия с внешней средой.
- **Нерасчлененность:** элементы системы находятся в отношениях взаимозависимости друг с другом.
- **Множественность описаний:** из-за высокой сложности системы её познание требует использования множеств различных взглядов и моделей.

5. Модели жизненного цикла разработки ПО.

Модель ЖЦ — это структура, определяющая последовательность выполнения и взаимосвязи процессов и задач на протяжении всего цикла. В материалах выделяются три **классических модели**:

1. **Каскадная (водопадная) модель:** характеризуется строго последовательным переходом по этапам. Следующий этап начинается только после полного завершения предыдущего.
2. **Потопная модель с итерационным контролем (итерационная):** в отличие от каскадной, допускает возвраты на предыдущие этапы при обнаружении несоответствий или ошибок в ходе контроля.
3. **Спиральная модель:** делает упор на анализ рисков. Разработка идет витками, на каждом из которых создается протокол системы, уточняются требования и планируется следующий виток. Также упоминаются современные стратегии конструирования: **однотактные** и **итерационные** (итерационные).

9. Организация разработки ИС. Стандартизация проектирования

Организация процесса разработки опирается на использование общепринятых стандартов, которые регламентируют жизненный цикл ПО. В материалах выделяется:

- **ГОСТ 34.601-90** — отраслевой стандарт, определяющий стадии создания автоматизированных систем.
- **ISO/IEC 12207:1995** — международный стандарт процессов жизненного цикла ПО.
- **Промышленные методики:** CDM (методика Oracle), RUP (Rational Unified Process), MSF (Microsoft Solution Framework) и XP (Extreme Programming).

Стандартизация позволяет формализовать переход от модели предметной области к физической реализации через создание **концептуальной, авторитарной** (база данных) и **внешней** (программный комплекс) моделей.

13. Методологии структурного анализа и проектирования. Семейство IDEF

IDEF (Integration Definition Methodology) — это семейство совместно используемых методов моделирования. Основные методы семейства:

- **IDEF0:** Функциональное моделирование. Используется для создания структурного отображения функций системы и объектов, их взаимодействии.
- **IDEF1:** Информационное моделирование. Применяется для построения модели структурной информации, необходимой для поддержки функций системы.
- **IDEF2:** Динамическое моделирование поведения функций и ресурсов (используется редко, в основном для компьютерных производств).
- **IDEF3:** Описание процессов (логики выполнения действий и переходов состояний объектов).
- **IDEF4:** Объектно-ориентированное планирование.
- **IDEF5:** Формирование онтологического описания системы (каталог терминов и правил).
- **IDEF14:** Моделирование вычислительных сетей.

2. Основные понятия информационной системы (ИС). Классы ИС.

Информационная система — это взаимосвязанная совокупность средств, методов и персонала, используемых для хранения, обработки и выдачи информации в интересах достижения поставленной цели. **Обеспечение безопасности ИС:** Для полноценного функционирования ИС включает в себя несколько видов обеспечения: **информационное** (системы классификации, схемы потоков данных), **техническое** (комплекс средств и документаций), **математическое и программное** (алгоритмы и программы), **организационное** (взаимодействие персонала) и **правовое** (юридический статус и нормы).

Классификация ИС:

- **По типу данных:** фактографические и документальные.
- **По степени автоматизации:** ручные, автоматизированные и автоматические.
- **По сфере применения:** интერнетные системы, системы организационного управления, системы управления технологическими процессами (ТП) и САПР.
- **По характеру обработки данных:** информационно-поисковые и информационно-решающие (которые, в свою очередь, делятся на управляющие и консультные).
- **По уровню управления:** стратегические, функциональные и операционные.

6. Методологии проектирования ИС

Методологии проектирования — это совокупность методов, приемов и средств, используемых в процессе создания информационной системы. В материалах выделяются следующие **основные характеристики** методологий:

1. **Стратегия конструирования:** согласно стандарту IEEE/EIA 12207.2, она может быть **однотактной** (традиционной) или **итерационной** (итерационной).
 2. **Адаптивность процесса:** способность методологии подстраиваться под нужды проекта.
 3. **Системный этап:** последовательность процессов и задач, выполняемых на протяжении всего ЖЦ.
 4. **Формулировка требований:** способы определения того, что должна делать система.
- В источниках приводится обширный список методологий, включая **Waterfall model** (каскадная), **Spiral model** (спиральная), **RUP**, **XP**, **Scrum**, **MSF**, а также **DSDM** (Dynamic System Development Method).
- Принципы DSDM** (как пример современной гибкой методологии):
- Активное участие пользователей и заказчиков в процессе разработки.
 - Самостоятельность команды в принятии решений.
 - Деление работы на **спринты** с демонстрацией итогового результата после каждого.
 - Использование итерационного и инкрементного подходов.
 - Постоянное тестирование и возможность «отката» любых действий назад.

10. Основные принципы проектирования ИС

В лекциях приводится перечень из **11 фундаментальных принципов**, которыми должен руководствоваться проектировщик:

1. **Декомпозиция:** разбиение сложной системы на более простые части по принципу «разделяй и властвуй».
2. **Нерасчлененное упорядочение:** организация частей системы в согласованные структуры.
3. **Концептуальная общность:** единство взглядов и подходов на всех этапах проектирования.
4. **Абстрагирование:** выделение существенных характеристик объекта, исключая несущественные.
5. **Формализация:** использование строгих методов и языков для описания системы.
6. **Унификация:** применение единообразных решений и стандартов.
7. **Визуальная независимость:** автономность частей системы друг от друга.
8. **Минимализация:** использование разных моделей (функциональных, информационных, поведенческих) для описания одной системы.
9. **Нерепротиворечивость:** согласованность всех элементов модели между собой.
10. **Информационная закрытость (инкапсуляция):** скрытие внутренней реализации модулей.
11. **Полнота:** возможность использования одного и того же интерфейса для различных типов данных или объектов.

14. Применение методологии IDEF2. Элементы графической нотации

IDEF2 предназначена для формализации и описания бизнес-процессов через функциональное моделирование. **Базовые принципы:**

1. **Функциональная декомпозиция:** разбиение сложных функций на более простые.
2. **Отграничение сложности:** на одной диаграмме должно быть от 2 до 6 блоков.
3. **Контекст:** наличие главной функции (диаграмма АВ), определяющей границы нотации.

Элементы нотации:

- **Функциональный блок:** Прямоугольник, обозначающий конкретную бизнес-функцию.
- **Стрелка (Идентифицирующая дуга):**
 - **Вход (левая сторона):** Материал или информация, которые преобразуются работой.
 - **Управление (верхняя сторона):** Правила, стратегии, стандарты. Каждый блок должен иметь хотя бы одну стрелку управления.
 - **Выход (правая сторона):** Результат выполнения функции.
 - **Механизм (нижняя сторона):** Ресурсы (персонал, станки, бюджет).
 - **Вызов (нижняя сторона, стрелка вниз):** Ссылка на другую модель для организации коллективной работы.

3. Понятие модели. Виды моделей. Задачи моделирования. Модели ИС.

Модель — это формальное описание особенностей системы, которые являются существенными для целей её исследования.

Виды моделей по способу представления:

1. **Вербальные** (словесные описания).
2. **Натурные** (физическое воспроизведение объекта).
3. **Знаковые** (включая математические модели, описывающие состояние и изменения системы на языке алгоритмов и формул).

Классификация моделей ИС:

- **По степени описания:** формальные и неформальные.
- **По степени физической реализации:** логические и физические.
- **По отображению данных:** статические и динамические.
- **По отображаемому аспекту:** функциональные, информационные, поведенческие, компонентные и смысловые.

Процесс формирования информационной модели проходит стадии от изучения системы до её моделирования и последующего сопровождения.

7. Подходы к выбору методологии для разработки ПО

Выбор методологии зависит от масштаба проекта и его критичности. В лекциях сформулированы следующие **принципы выбора**.

1. **Масштаб команды:** большая по размерам методология требуется тогда, когда в проекте много большое число разработчиков.
2. **Критичность системы:** «нотации» (строго регламентированная) методология необходима в случаях, когда ошибки в ПО могут повлечь непоправимый ущерб.
3. **Сложность проекта:** следует учитывать, что даже незначительное увеличение сложности методологии ведет к существенному росту стоимости проекта.
4. **Эффективность коммуникации:** наиболее эффективной формой передачи информации считается непосредственное взаимодействие «лицом к лицу».

4. Жизненный цикл и стадии разработки программного обеспечения (ПО).

Жизненный цикл ПО (ЖЦ ПО) — это период времени, который начинается с момента принятия решения о необходимости создания ПО и заканчивается в момент его полного снятия из эксплуатации.

Основные этапы ЖЦ ПО:

1. **Формирование требований к системе (планирование):** определение того, что должна делать система.
2. **Проектирование:** разработка архитектуры и структуры.
3. **Реализация (кодирование):** написание программного кода.
4. **Тестирование:** проверка работоспособности и поиск ошибок.
5. **Выход в действие:** передача системы заказчику.
6. **Эксплуатация и сопровождение:** поддержка системы в рабочем состоянии.

Для каждого этапа строго определяется состав работ, получаемые результаты, роли участников и используемые методы. ЖЦ регламентируется стандартами, такими как **ГОСТ 34.601-90**, **ISO/IEC 12207:1995**, а также методологиями **RUP**, **MSF** и **XP**.

8. Базовые требования ИС: каскадное планирование и техническое задание

Проектирование информационной системы рассматривается как **процесс перехода** от первичного описания к детальной документации:

- **Техническое задание (ТЗ):** это первичная модель, представляющая собой исковое описание системы. На основе ТЗ в процессе проектирования создается набор стандартизированных документов (проектная документация), достаточных для создания ИС.
- **Планирование:** это первый этап жизненного цикла ПО, на котором происходит формирование требований к системе.

Для каждого этапа (включая планирование) formally определяются:

- Состав и последовательность работ.
- **Получаемые результаты** (отчеты, модели, документы).
- Роли и ответственность участников проектной команды.

11. Подходы к проектированию ИС. Основные концепции каждого подхода

В современной практике проектирования выделяют два основных подхода: **структурный** и **объектно-ориентированный (ООП)**.

- **Структурный подход:**
 - Система рассматривается как набор функций (**функциональная декомпозиция**).
 - Данные (атрибуты) хранятся и обрабатываются отдельно от функций (сущностей).
 - Иерархия модулей строится на основе отношений композиции, а сама структура системы представляется в виде **дерева**.
- **Объектно-ориентированный подход:**
 - Система представляется как набор взаимодействующих объектов (**объектная декомпозиция**).
 - Объект объединяет в себе данные (характеристики) и поведение (методы).
 - Иерархия строится на отношения композиции и наследования, а структура модуля представляется в виде **ориентированного графа**.
 - Преимуществом ООП является лучшее соответствие описания системы смыслу предметной области.

12. Структурный подход к проектированию ИС. История развития и сущность

История: Технологии структурного анализа и проектирования (IDEF) начала активно использоваться с конца 1980-х годов. Основным заказчиком и покровителем выступило Министерство обороны США, а также крупные американские корпорации.

Сущность подхода:

- **Проектирование** — это процесс перехода от первичного описания (Технического задания) к набору стандартизированных документов (проектной документации), достаточных для создания ИС.
- Система разбивается на функциональные блоки, связанные потоками информации.
- Основным инструментом являются методологии семейства **IDEF** и диаграммы потоков данных (**DFD**).

16. Применение методологии IDEF3 для моделирования предметной области. Элементы графической нотации IDEF3.

IDEF3 — это методология, предназначенная для описания процессов, происходящих в системе, и фиксации логики выполнения действий. Она может использоваться как самостоятельно, так и в дополнение к IDEF0 для детализации её функциональных блоков.

Метода в системе IDEF3:

1. **FPD (Process Flow Description):** описание технологических процессов с указанием того, что происходит на каждом этапе.
2. **OSTD (Object State Transition Description):** описание переходов состояний объектов и их промежуточных состояний.

Элементы нотации:

- **Действия (Units of Work):** изображаются прямоугольниками с именем и номером.
- **Связи (стрелки):**
 - **Временное прерывание:** исходное действие должно завершиться перед началом конечного.
 - **Объектный поток:** выход одного действия является входом для другого.
 - **Нечеткое отношение:** связь, логика которой задается аналитическим индивидуальным.
- **Соединения (перекрестки):** используются для описания ветвления процессов. Былают развешивающимися (одно действие вызывает несколько) и сходящимися (несколько действий предшествуют одному).
- **Типы логики перекрестков:** «И» (&), «ИЛИ» (O) и «Исключающее ИЛИ» (X).
- **Вспомогательные элементы:** Указатель «ОБЪЕКТ» (участие объекта в действии) и «УТОЧНЕНИЕ» (подробное описание логики ветвления).

17. Методологии функционального моделирования. Применение методологии DFD для описания потоков данных предметной области. DFD (Data Flow Diagram) — это диаграмма потоков данных, представляющие итерацию процессов, связанных потоками информации. В отличие от IDEFO, DFD фокусируется на передаче данных между процессами и накопителями.

Основные компоненты DFD:

- Внешние сущности:** материальные объекты или лица (источники/приемники информации), изображаются в виде прямоугольника с тенью.
- Процессы (работы):** функции системы, преобразующие входные потоки в выходные по заданному алгоритму.
- Накопители данных (хранилища):** места хранения информации.
- Потоки данных:** стрелки, описывающие движение объектов и информации; могут быть двунаправленными для диалогов «команда-ответ».

Основные правила построения:

- Все потоки данных между хранилищами должны проходить через процесс.
- Нотация не содержит логических операторов.
- Все элементы должны иметь уникальные имена и быть описаны в словаре данных.
- Для стрелок не важно, в какую грань блока они входят или выходит.
- Построение всегда начинается с создания контекстной диаграммы уровня «черного ящика».

Дайте расшифровку терминам DFD, IDEF, IADT.

DFD (от англ. data flow diagram — диаграмма потоков данных) — это нотация, предназначенная для моделирования информационной систем с точки зрения хранения, обработки и передачи данных.

IDEF (I-CAM DEFinition or Integrated DEFinition) — методологии семейства I-CAM (Integrated Computer-Aided Manufacturing) для решения задач моделирования сложных систем позволяют отображать и анализировать модели деятельности широкого спектра сложных систем в различных вариантах.

SADT (Structured Analysis and Design Technique) — это методологии структурного анализа и проектирования. Эта методология разработана для описания и понимания функционирования искусственных систем. Основные элементы SADT-модели: блоки, обозначающие функции, и стрелки (дуги), обозначающие объекты (входы и выходы функций).

18. Понятие CASE-технологии и её составляющие. CASE-инструменты. Инструментальные программные средства для построения и визуализации моделей предметной области. CASE-технология (Computer-Aided Software Engineering) представляет собой совокупность методов и инструментов для автоматизации проектирования и разработки ПО. Они позволяют визуализировать структуру системы через итерацию моделей.

Инструментальные средства (CASE-инструменты), унавоимые в лекциях:

- Капис:** отечественное средство моделирования.
- BPwin (BPMision Process Modeler):** классический инструмент для IDEFO, IDEF2 и DFD.
- CASE-технология (Computer-Aided Software Engineering):** предоставляет собой совокупность методов и инструментов для автоматизации проектирования и разработки ПО. Они позволяют визуализировать структуру системы через итерацию моделей.
- MS Visio и DrawVis:** универсальные графические редакторы для построения диаграмм.
- Lucidchart:** облачное решение для совместной командной работы над моделями.

Эти средства обеспечивают ясность описания системы на каждом уровне детализации и позволяют разбить её на части с точно определенными отношениями.

19. Объектно-ориентированное моделирование. Основные понятия в объектно-ориентированном подходе (ООП). Базовые составляющие ООП. Сущность объектно-ориентированного подхода (ООП) заключается в декомпозиции системы на классы, которые соответствуют реальным объектам предметной области. В отличие от структурного подхода, где данные и функции разделены, в ООП они объединяются внутри объекта.

Базовые понятия и составляющие:

- Класс:** шаблон для создания объектов, описывающий их свойства и поведение. На диаграммах изображается прямоугольником, разделенным на три части: имя, атрибуты и операции.
- Объект:** конкретный экземпляр класса.
- Структура системы:** представляется в виде ориентированного графа, где вершины строятся на отношениях композиции и наследования.
- Отношение между элементами:**
 - Зависимость (dependency):** изменение одного элемента может повлиять на другой.
 - Ассоциация:** связь между классами с указанием кратности (например, 1, 0..1, *).
 - Обобщение (generalization):** отношение наследования от предка к потомку.
- Ключевые принципы (из общих принципов проектирования):** инкапсуляция (закрытость данных), наследование и полиморфизм.

20. Методологии, поддерживающие ООП. Унифицированный процесс. Для реализации объектного подхода используются специализированные методологии, такие как RUP (Rational Unified Process), MSF и XP (Extreme Programming).

Унифицированный процесс (Unified Process):

- Поддерживает построение нескольких видов диаграмм для описания разных аспектов системы.
- Для сложных систем строится множество диаграмм одного и того же вида для разной степени детализации.

Режимы использования моделей в разработке:

- Режим эскиза:** наброски для обсуждения идей.
- Режим проектирования:** детальные модели, включающие прямую (forward-engineering) и обратную (reverse-engineering) разработку кода.
- Режим языка программирования:** исполняемый UML (Executable UML).

Также упоминаются концепции MDA (Model Driven Architecture), которая разделяет платформу-независимые модели (PIM) и платформу-зависимые модели (PSM).

21. Основы унифицированного языка моделирования UML. История создания и назначение языка. Режимы использования.

UML (Unified Modeling Language) — это унифицированный язык моделирования, предназначенный для описания, визуализации и документирования объектно-ориентированных систем.

История создания: Авторами языка являются ведущие специалисты в области ООП — Грейд Юн, Джеймс Рамбо и Аллан Джексон. Язык создавался с целью объединения различных подходов к объектному моделированию в единый стандарт.

Назначение: Описание типов объектов в системе и различных статических отношений между ними.

Режимы использования UML:

- Режим эскиза:** создание набросков для быстрого обсуждения идей.
- Режим проектирования:** детальные моделирование, включающее **прямую разработку** (forward-engineering — генерация кода по модели) и **обратную разработку** (reverse-engineering — восстановление модели из кода).
- Режим языка программирования:** использование исполняемого UML (Executable UML).

22. Структура UML. Семантика и синтаксис UML. Понятие нотации. Понятие UML-диаграммы. Взаимосвязи между диаграммами.

Понятие нотации и синтаксиса: UML использует графическую нотацию для представления элементов. Например, класс в нотации изображается прямоугольником, разделенным на три части: имя класса, его атрибуты и его операции.

Семантика: Описывает значение элементов и связей. К основным видам связей относятся:

- Зависимость (dependency):** изменение в одном элементе может вызвать изменение в другом.
 - Ассоциация:** непрерывная линия, указывающая на наличие связи между классами.
 - Обобщение (generalization):** отношение наследования, где все свойства «потомка» справедливы для «наследника».
- UML-диаграммы: Это графическое представление модели ИС с определенной точки зрения (например, функциональной или поведенческой).
- Взаимосвязи между диаграммами: В рамках Унифицированного процесса (Unified Process) для одной модели системы строятся несколько видов диаграмм. Диаграммы взаимосвязаны: части диаграмм после их построения часто требуют развития и уточнения в рамках последующих моделей.

23. Классификация UML-диаграмм. Виды моделей ИС и их связь с UML-диаграммами.

Стандарт UML 2 включает в себя 13 официальных типов диаграмм, которые делятся на две большие группы: **структурные** и **диаграммы поведения**.

- Структурные диаграммы (Static):**
 - Классы:** классы, свойства и отношения (функционально-информационный аспект).
 - Объекты:** конфигурация контекста, экземпляров.
 - Пакеты:** иерархическая структура времени композиции (компонентный аспект).
 - Компоненты:** структура программных компонентов.
 - Развертывание:** физическая реализация артефактов на узлах (физический аспект).
- Диаграммы поведения (Dynamic):**
 - Варианты использования:** функции системы (функциональный аспект).
 - Деятельность:** бизнес-процессы и алгоритмы поведения (поведенческий аспект).
 - Конечных автоматов:** изменения объекта в течение жизни (поведенческий аспект).
- Диаграммы в взаимодействии:** последовательности, коммуникации, обмена взаимодействиями и временные (акцент на передаче сообщений и синхронизации).

24. Диаграмма поведения. Общие сведения, назначение и взаимодействие поведенческих диаграмм.

Диаграммы поведения (Behavior Diagram) описывают то, как система функционирует и взаимодействует с пользователями или внешней средой.

Основные типы:

- Диаграмма прецедентов (Use Case):** определяет функциональное назначение (что делает система).
 - Диаграмма деятельности (Activity):** описывает логику процесса, потоки работ и параллельное поведение.
 - Диаграмма состояний (State Machine):** отображает состояния сущности и переходы между ними под влиянием событий (триггеров).
 - Диаграммы взаимодействия:** фокусируются на обмене сообщениями между объектами. Например, **диаграмма последовательности** (Sequence diagram) подчеркивает временную упорядоченность сообщений.
- Взаимодействие: Эти диаграммы дополняют друг друга: прецеденты задают общие требования, деятельность детализирует алгоритм их выполнения, а диаграммы состояний описывают внутреннюю логику отдельных объектов.

25. Назначение и состав диаграммы вариантов использования. Примеры.

Диаграмма вариантов использования (Use Case diagram) описывает функциональные возможности системы без раскрытия её внутренней структуры.

Назначение: спецификация внешних требований в системе и определение того, как она будет использоваться.

Состав нотации:

- Актеры (Actors):** роли, которые пользователи или другие системы играют по отношению к ИС.
- Прецеденты (Use Cases):** функциональные фрагменты поведения (функции).

Связи:

- Ассоциация:** связь Актера и Прецедента.
- Обобщение:** например, роль «Менеджер» может наследовать свойства роли «Пользователь».
- Включить (include):** один прецедент обязательно включает поведение другого.
- Расширить (extend):** прецедент может быть дополнен в определенных условиях.

Пример: Для системы аналитики соцсетей выделяются актеры «Пользователь» и «Менеджер», а также прецеденты: «Ввод запроса на анализ», «Получение данных», «Просмотр отчета», «Аутентификация».

26. Назначение и состав диаграммы деятельности. Примеры нотаций.

Диаграмма деятельности (Activity diagram) — это нотация, предназначенная для описания логики процедур, бизнес-процессов и потоков работ. Она позволяет визуализировать параллельно и процедурное поведение системы.

Состав нотации:

- Начальный узел (initial node):** точка начала выполнения процесса.
 - Операции (actions):** основные шаги или действия в рамках деятельности.
 - Ветвление (fork) и Объединение (join):** используются для описания параллельно выполняемых потоков работ.
 - Решение (decision) и Слияние (merge):** используются для описания условного ветвления процесса (логика «если... то»).
 - Конечный узел (final node):** завершение конкретного потока или всей деятельности.
 - Рубца (swimlane):** стрелки, соединяющие узлы и показывающие последовательность действий.
- Пример: В лекциях приведена диаграмма процесса доставки заказа («Deliver Order»), где после принятия заказа поток разделяется на две ветки: срочная доставка («Rush Order») и обычная доставка («Regular Delivery»), которые затем сливаются в общий узел завершения.

27. Диаграмма состояний. Назначение и состав диаграммы. Примеры нотаций.

Диаграмма состояний (State Machine diagram) описывает жизненный цикл одной сущности, показывая, как события изменяют объект в течение его жизни.

Состав нотации:

- Начальное псевдосостояние (initial pseudostate):** точка входа в жизненный цикл.
- Состояния (states):** условия или ситуации в жизни объекта, во время которых он выполняет определенную деятельность.
- Переход (transition):** переключение из одного состояния в другое под влиянием триггера.
- Конечное состояние (final state):** завершение жизненного цикла объекта.

Метки переходов и активности:

- Каждый переход** может иметь метку: **Триггер (Trigger) / Активность** («событие [сложное условие] / действие»).
- Внутри состояния** могут быть прописаны внутренние активности: **entry** (когда), **do** (выполняемая в состоянии), **exit** (когда).
- Переходы** бывают **триггерными** (по событию) и **внутригетерными** (связываемые по завершении внутренних операций до).

28. Диаграммы взаимодействия. Назначение и состав диаграммы последовательности. Примеры нотаций.

Диаграммы взаимодействия (Interaction diagram) фокусируются на обмене сообщениями между объектами.

Диаграмма последовательности (Sequence diagram):

- Назначение:** отображение временной упорядоченности передачи сообщений между объектами и акторами.
- Состав:**
 - Участник:** объект или актер, участвующий во взаимодействии.
 - Линия жизни:** вертикальная пунктирная линия, показывающая время существования объекта.
 - Активация:** узкий прямоугольник на линии жизни, показывающий, что объект выполняет операцию.
 - Сообщение:** горизонтальная стрелка. Бывают **синхронные** (вызывающий ждет ответа) и **асинхронные** (вызывающий продолжает работу, не дожидаясь ответа).

29. Структурные диаграммы. Общие сведения, назначение и взаимодействие структурных диаграмм.

Структурные диаграммы (Static) описывают типы объектов системы и различные статические отношения между ними. В отличие от диаграмм поведения, они не показывают динамику изменений во времени, а фиксируют архитектуру системы.

Основные сведения и типы:

- Диаграмма классов:** описывает логическую структуру (классы, свойства, отношения).
- Диаграмма пакетов:** показывает иерархическую структуру системы во время композиции.
- Диаграмма компонентов:** отображает разбиение системы на программные модули, библиотеки и зависимости между ними.
- Диаграмма развертывания:** описывает физическую реализацию артефактов на узлах (серверах, рабочих станциях).

Взаимодействие: Эти диаграммы дополняют друг друга в рамках Унифицированного процесса: классы детализируют внутреннее устройство пакета, а компоненты и развертывание показывают, как эти логические структуры воплощаются в физическом мире.

30. Диаграммы классов объектов. Назначение и состав диаграмм. Примеры.

Диаграмма классов (Class diagram) — центральное ядро объектно-ориентированного проектирования.

- Назначение:** описание типов объектов и их статических взаимосвязей.
- Состав нотации:** классы, атрибуты (характеристики) и операции (методы).
- Виды связей:**

- Зависимость (dependency):** изменение одного элемента может повлиять на другой.
- Обобщение (generalization):** наследование свойств родительского класса наследником.
- Ассоциация:** связь между классами с указанием **кратности** (например, 1, 0..1, *).
- Агрегация и Композиция:** типы ассоциаций «целое-часть».

Диаграммы объектов (Object diagram) — это вариант конфигурации конкретных экземпляров классов в определенный момент времени. Они используются для иллюстрации сложных структур классов на конкретных примерах.

Пример: В лекциях приведена диаграмма классов системы заказов, включающая классы «Order» (Заказ), «Customer» (Клиент) и его подклассы «Corporate Customer» и «Personal Customer», а также отношения между ними с указанием ролей и кратности.

31. Диаграмма пакетов. Назначение и состав диаграмм. Примеры нотаций.

Диаграмма пакетов (Package diagram) относится к группе структурных диаграмм UML.

Назначение: отображение иерархической структуры системы на этапе композиции. Она используется для группировки элементов модели (классов, других пакетов) в более крупные блоки для упрощения сложностей.

Состав и нотация:

- Пакет (Package):** изображается в виде панели с закладкой. Внутри прямоугольника указывается имя пакета.
- Полностью определенное имя:** может записываться в формате имя.дискет.имя_элемента (например, java:lang.Date).
- Взаимосвязь:** пакеты могут содержать внутри себя другие пакеты или перечни классов.
- Зависимость (Dependency):** изображается пунктирной стрелкой, указывающей, что один пакет использует ресурсы другого.
- Связь с моделью:** в нотации моделей ИС диаграмма пакетов отвечает за **компонентный аспект** и может быть как логической, так и физической.

32. Назначение и состав диаграммы компонентов. Примеры нотаций.

Диаграмма компонентов (Component diagram) — это структурная диаграмма, описывающая физическую реализацию программной системы.

Назначение: показывать разбиение программной системы на структурные компоненты и физические связи (зависимости) между ними. Она отображает, из каких модулей, библиотек и исполняемых файлов состоит система.

Состав нотации:

- Компонент (Component):** в стандарте UML 1 изображался как прямоугольник с двумя выступающими слева зубцами; в UML 2 — как прямоугольник с небольшой шпигетерной компонентой в правом верхнем углу.
- Зависимость:** стрелки, показывающие, как компоненты взаимодействуют друг с другом.

Пример: компонентом может быть файл `commons.dll` или `web-tools.jar`.

33. Модель физической реализации. Назначение и состав диаграммы реализации. Примеры нотаций. Диаграмма реализации (Deployment diagram) описывает физическую архитектуру системы.

- **Назначение:**
 1. Определение распределения программных компонентов по физическим узлам.
 2. Отображение физических связей между узлами реализации на этапе исполнения.
 3. Выявление «узких мест» системы и реинженеринг её топологии.
- **Состав нотации:**
 - Узел (Node): изображается в виде трехмерного куба. Обозначает вычислительный ресурс (сервер, рабочая станция).
 - Артефакт (Artifact): конкретная физическая сущность (программный компонент), которая создается или используется в процессе работы ПО (например, web-app.war или UserServices.cs). Изображается прямоугольником с ключевым словом «artifact».
 - Соединение: линии, показывающие физические каналы связи между узлами.

37. Модульная архитектура. Виды декомпозиции и их возможности. Модульная архитектура — это способ организации системы, при котором она разбивается на независимые, заменяемые части (модули) с четко определенными интерфейсами.

- Виды декомпозиции (согласно лекциям):**
1. **Функциональная декомпозиция (Структурный подход):**
 - Система разбивается на набор функций (подпроцессов).
 - Данные хранятся отдельно от функций.
 2. **Объектная декомпозиция (Объектно-ориентированный подход):**
 - Система разбивается на набор взаимодействующих объектов, объединяющих данные и поведение.
 - Иерархия строится на отношениях композиции и наследования.
 - Структура представляется в виде **ориентированного графа**.
- Возможности декомпозиции:** позволяет управлять сложностью, обеспечивает параллельную разработку модулей разными командами и упрощает тестирование системы.

34. Механизмы расширения UML диаграмм. Основные подходы и методы. UML является гибким языком и включает механизмы для уточнения и расширения стандартной нотации.

- **Примечания (Notes):** примечательным с заголовком углом, содержащий произвольный текст. Используется для комментариев разработчика, ограничений или пояснений.
- **Стереотипы:** позволяют расширить семантику элемента. Используются в углах кадрах, например, «interface» или «extend» на диаграмме прецедентов, или «artifact», «library», «node» на структурных диаграммах.
- **Ограничения и стереотипы условия (Stereos):** текст в фигурных скобках [...] или выделенный шрифтом. Например, условие перехода на диаграмме состояний: [штатка не в наличии].
- **Помеченные значения:** пары «имя = значение», позволяющие добавлять новые свойства элементам модели (например, «участок = 1.01»).

38. Понятие «типовое решение». Типовое проектное решение. Архитектурные шаблоны.

- **Типовое решение:** это прообраз на практике комбинация методов и средств, которая может быть повторно использована для решения схожих задач.
- **Типовое проектное решение (TPR):** формализованный набор проектной документации и программных кодов, предназначенный для массового применения. В лекциях упоминаются «передовые практики» (Best Practices), которые кажутся для эффективного руководства работами и принятия решений.
- **Архитектурные шаблоны:** это схемы самого высокого уровня, определяющие фундаментальную структуру системы. В источниках упоминаются такие модели, как **Waterfall, Spiral, RUP, IP** и **MDA** (архитектура, управляемая моделью).

35. Архитектура ИС. Классификация ИС по их архитектуре. В лекционных материалах архитектура рассматривается через призму моделей и методологий проектирования.

- **Модельно-ориентированный подход (MDA):** выделяет два уровня архитектуры:
 1. **PM (Platform Independent Model):** архитектура, не зависящая от конкретной технологической платформы.
 2. **PSM (Platform Specific Model):** архитектура, привязанная к конкретной платформе (например, Java или .NET).
- **Физическая архитектура (по диаграммам развертывания):** классифицируется по распределению артефактов по узлам (серверная часть, клиентская часть, база данных).
- **Структурная организация:** может быть представлена как **иерархия функций** (структурный подход, где данные отделены от функций) или как **граф взаимодействующих объектов** (OOI, где данные и методы объединены).

Примечание: детальная классификация на «тонкой/толстой клиент», которая упоминается в курсе 36, в данных фрагментах не приводится, но используется контекстом используемых веб-ориентированных систем.

39. Паттерны проектирования. Виды паттернов проектирования. Основные цели и задачи их применения.

- Дополнительная информация (в лекциях паттерны упоминаются контекстно через UML, но не классифицируются).*
- Паттерны проектирования** — это повторяемая архитектурная конструкция, представляющая собой решение типичной проблемы в рамках конкретного контекста.
- Классификация паттернов (по GoF):**
1. **Порождающие (Creational):** механизмы создания объектов (например, Singleton, Factory).
 2. **Структурные (Structural):** определяют отношения между классами и объектами (например, Adapter, Facade).
 3. **Поведенческие (Behavioral):** управляют алгоритмами и взаимодействием объектов (например, Observer, Strategy).
- Цели применения:** унификация кода, повышение его читаемости, упрощение повторного использования и ускорение процесса проектирования.

36. Базовые архитектурные шаблоны ИС (на примере веб-ориентированных систем).

- Архитектура тонкого клиента, толстого клиента и веб-доставки. Архитектура ИС определяет распределение компонентов по узлам и способы их взаимодействия.
- **Тонкий клиент (Thin Client):**
 - Почти вся логика приложения и управление данными сосредоточены на **сервере**.
 - Клиентская часть отвечает только за отображение интерфейса и ввод данных (чаще всего это браузер).
 - **Преимущества:** простота обновления (только на сервере), низкие требования к клиентскому оборудованию.
 - **Толстый клиент (Thick/Pat Client):**
 - Значительная часть бизнес-логики и иногда хранение временных данных перенесены на **сторону клиента**.
 - Сервер используется в основном для хранения центральной базы данных и тяжелых вычислений.
 - **Преимущества:** высокая интерактивность, возможность работы офлайн, снижение нагрузки на сервер.
 - **Веб-доставка (Web Delivery):**
 - Современный подход, при котором клиентское приложение (артефакты, такие как web-app.war) динамически загружается в браузер или среду исполнения. Это позволяет сочетать гибкость «тонкого клиента» с функциональностью «толстого» (например, через технологию SPA — Single Page Application).

40. Архитектурные шаблоны MV*. Дополнительная информация (отсутствует в явной виде в предоставленных материалах).

- Семейство шаблонов **MV+ (Model-View-*)** предназначено для разделения бизнес-логики и пользовательского интерфейса.
- **MVC (Model-View-Controller):** классическая схема. Model хранит данные, View отображает их, Controller обрабатывает ввод пользователя и обновляет модель.
 - **MVP (Model-View-Presenter):** View полностью пассивен. Presenter выступает посредником, забирая данные из модели и передавая их во View.
 - **MVVM (Model-View-ViewModel):** используется в современных веб-и мобильных фреймворках. **View-Model** обеспечивает двустороннюю привязку данных (Data Binding) между представлением и моделью.
- Основная цель:** «разделение ответственности» (Separation of Concerns), что облегчает тестирование и модификацию ИС.

41. Архитектура корпоративных программных решений. Современные подходы к организации ИС. ПРИНЦИП функциональной декомпозиции. ПРИНЦИП объектно-ориентированной декомпозиции. ПРИНЦИП унифицированного процесса (Unified Process).

- **Концепция MDA:** Это архитектура, управляемая моделью. Она предполагает разделение логики работы системы и её технической реализации. Выделяются уровни:
 - **PM (Platform Independent Model):** модель, не зависящая от платформы. Она описывает бизнес-логику системы в чистом виде.
 - **PSM (Platform Specific Model):** Модель, зависящая от конкретной платформы (например, Java, .NET). Она получается путем трансформации PM под требования среды исполнения.
 - **Executable UML (исполнимый UML):** Позволяет генерировать работоспособный код напрямую из моделей.
- **Унифицированный процесс (UP):** Обеспечивает упорядоченный подход в разработке, где для одной модели системы строятся сразу несколько видов диаграмм (Use Case, Class, Sequence и др.), что позволяет описать архитектуру с разных точек зрения.
- **Принципы организации:** Проектирование корпоративных систем опирается на принципы **лической независимости**, **инкапсуляции** (информационной закрытости) и **многомодальности**.

42. Архитектура микросервисов (Microservices Architecture). Дополнительная информация (отсутствует в лекциях): Микросервисная архитектура — это подход к разработке ПО как набору небольших независимых сервисов, каждый из которых работает в собственном процессе и взаимодействует с другими с помощью легковых механизмов (обычно HTTP/REST или очереди сообщений).

- **Ключевые характеристики:**
 1. **Децентрализация:** Каждый сервис имеет свою базу данных и может быть написан на своем языке программирования.
 2. **Независимое развертывание:** Обновление одного сервиса не требует перезапуска всей системы.
 3. **Импляция сбоя:** Ошибка в одном компоненте не приводит к падению всей системы (при правильном проектировании).
- **Связь с UML (на основе материалов):** В такой архитектуре критически важны **диаграммы компонентов** (для описания связей между сервисами) и **диаграммы развертывания** (для описания физического распределения сервисов по узлам контейнера).

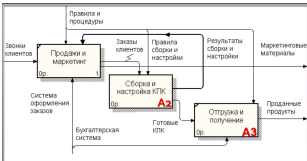
43. Бессерверная архитектура (Serverless Architecture). Дополнительная информация (отсутствует в лекциях): Serverless — это модель облачных вычислений, при которой разработчик пишет код (функции), а облачный провайдер полностью берет на себя управление инфраструктурой (серверами), масштабированием и выделением ресурсов.

- **Основные концепции:**
 1. **FaaS (Function as a Service):** Код выполняется только в ответ на конкретные события (триггеры).
 2. **Объектно-ориентированная модель:** Система реагирует на запросы файла, HTTP-запрос или сообщение в очередь.
 3. **Оплата по факту:** Ресурсы оплачиваются только в момент исполнения функции.
- **Проектирование:** Для описания логики таких систем в UML наиболее эффективны **диаграммы деятельности** (Activity) для визуализации потоков работ и **диаграммы состояний** (State Machine) для описания переходов функций под влиянием внешних триггеров.

44. Понятие «рефакторинг». Методы рефакторинга. В предоставленных источниках термин «рефакторинг» в контексте кода не упоминается, однако описывается процесс модификации кода при переходе от состояния AS-IS (как есть) к TO-BE (как должно быть).

- Дополнительная информация (отсутствует в лекциях):** Рефакторинг — это процесс изменения внутренней структуры программного обеспечения без изменения его внешнего поведения. Цель — сделать код более понятным, чистым и легким для сопровождения.
- **Основные методы рефакторинга:**
 1. **Extract Method (Выделение метода):** Если фрагмент кода в одном методе слишком длинный или сложный, он выносится в отдельный метод.
 2. **Копирование (Перемещение):** Перенос переменных, классов и методов имен, точно отражающих их суть.
 3. **Encapsulate Field:** Скрытие полей данных за методами доступа (getterами и setterами).
 4. **Move Method/Field:** Перенос функционала в тот класс, где он используется чаще всего.
 - **Связь с принципами проектирования (из лекций):** Рефакторинг напрямую направлен на достижение принципов **лической независимости**, **унификации** и **концептуальной общности**, описанных в материалах.

IDEP0 — методология функционального моделирования и графическая нотация, предназначенная для формализации и описания бизнес-процессов. ПРИНЦИП функциональной декомпозиции. 2. принцип ограничения сложности. 3. принцип контекста. ДИАГРАММЫ СОДЕРЖАТ блоки, стрелы и правила. Блоки представляют функции, описываемые как деятельность, процесс, описание, действие или преобразование. Стрелки представляют данные или материальные объекты, связанные с функциями. Правила определяют, как следует применять компоненты, диаграмма обеспечивает формат графического и словесного описания моделей. ВИДЫ СТРЕЛОК: «вход» управление «выход» механизм «вызов»



IDEP3 — методология описания процессов, происходящих в системе. IDEP3 может использоваться самостоятельно или вместе с методологией IDEP0. Процесс Flow Description (FD) — Описание технологических процессов, с указанием того, что происходит на каждом этапе технологического процесса. 2. Object State Transition Description (OSTD) — описание переходов состояний объектов, с указанием того, какие существуют промежуточные состояния у объектов в моделируемой системе. Каждая работа в IDEP3 описывает какой-либо линейный бизнес-процесс и может являться составляющей другой работы. Компоненты диаграммы описания процесса: Работы (boxes, activities) • Связи (стрелы — arrows, links) • Переходы (junctions) • Объекты состояний • Единичия Поведения (Unit of Behavior) • Разложение (Decomposition) • Разработчик (Elaboration) ТИПЫ СОЕДИНЕНИЙ: и каждое, «эксклюзивное или» одно и только одно, «или» одно или несколько, все выполняющиеся и разворачивающиеся



ДИАГРАММА ВАРИАНТОВ ИСПОЛЬЗОВАНИЯ АКТЕРЫ ПРЕДСТАВЛЯЮТ ПОДСИСТЕМА СВЯЗИ АКТЕР-ПРЕДСТАВЛЕНИЕ СООБЩЕНИЯ, ПРЕДСТАВЛЕНИЕ ПРЕДСТАВЛЯЕТ ИНТЕРФЕЙС, АКТЕР-АКТЕРОПОБЫЩЕНИЕ

Диаграмма вариантов использования описывает функциональное назначение системы

ДИАГРАММА КЛАССОВ Диаграмма описывает типы объектов системы и различного рода статические отношения, которые существуют между ними. Применительно к диаграмме представляются классы и разделены на три части: «имя класса», его атрибуты и его операции. ВИДЫ СВЯЗЕЙ НА ДИАГРАММЕ: КЛАССОВ: зависимость, обобщение, ассоциация, агрегация и композиция

